



山西大学
SHANXI UNIVERSITY

2026 届本科生毕业论文（设计）

题 目	面向虚拟仿真环境的机器人抓取系 统设计与实现
--------	---------------------------

学 院	计算机与信息技术学院
--------	------------

专 业	计算机科学与技术
--------	----------

姓 名	谢荣
--------	----

学 号	202201001160
--------	--------------

学习年限	2022 年 9 月至 2026 年 7 月
------	------------------------

指导教师	陈路
------	----

申请学位	工学学士学位
------	--------

二〇二六年六月

虚拟仿真环境机器人抓取系统设计与实现

摘要：随着机器人技术与计算机视觉的发展，基于视觉的抓取方法逐渐成为研究热点，但多数研究工作依赖于真实环境。基于这一情况，本文围绕在虚拟环境下的机器人抓取仿真展开研究。在Unity平台中构建三维仿真环境，采用YOLO模型对RGB图像数据进行处理，实现对目标物体的实时检测与定位，平均识别率达到90%，帧率达到10fps；使用GraspNet模型对深度图产生的目标点云数据进行处理，实现对目标物体的候选抓取姿态生成，平均生成姿态达到100个，候选姿态达到50个，有效姿态约1~3个。最终将筛选处理后的数据与机械臂运动控制模块进行集成，实现在虚拟环境中的闭环抓取仿真。本研究相比于真实环境成本更具优势，验证了虚拟仿真技术在机器人抓取中的可行性，也向实际的系统过渡提供了一定参考。

关键词：虚拟仿真；机器人抓取；目标检测；抓取位姿估计

Design and Implementation of a Robotic Grasping System in a Virtual Simulation Environment

ABSTRACT : With the development of robot technology and computer vision, vision-based grasping methods have gradually become a research hotspot, but most of the research work relies on the real environment. Based on this situation, this paper focuses on the research of robot grasping simulation in virtual environment. A three-dimensional simulation environment is constructed in the Unity platform, and the YOLO model is used to process the RGB image data to realize the real-time detection and positioning of the target object. The average recognition rate reaches 90 %, and the frame rate reaches 10 fps. The GraspNet model is used to process the target point cloud data generated by the depth map to generate the candidate grasping posture of the target object. The average generated posture reaches 100, the candidate posture reaches 50, and the effective posture is about 1 ~ 3. Finally, the filtered data is integrated with the manipulator motion control module to realize the closed-loop grasping simulation in the virtual environment. Compared with the real environment cost, this study has more advantages, which verifies the feasibility of virtual simulation technology in robot grasping, and also provides a certain reference for the transition to the actual system.

Keywords: Virtual simulation; robot grasping; target detection; grasping pose estimation

目录

1 引言	1
1.1 虚拟仿真抓取系统研究背景	1
1.2 虚拟仿真抓取系统研究目的与意义	1
2 虚拟仿真抓取系统研究	3
2.1 虚拟仿真抓取系统研究现状	3
2.2 虚拟仿真抓取系统抓取算法研究现状	4
2.2.1 常见抓取算法方式	4
2.2.2 抓取算法研究现状	5
2.3 虚拟仿真抓取系统逆运动学算法研究现状	6
2.3.1 常用逆运动学方法	6
2.3.2 Unity中的逆运动学方法	9
3 虚拟仿真抓取系统实现	10
3.1 虚拟仿真抓取系统结构分析	10
3.2 虚拟仿真抓取系统研究路线	10
3.3 虚拟仿真抓取系统研究方法	11
3.4 虚拟仿真抓取系统设计方案	14
3.4.1 机械臂的模型实现	14
3.4.2 目标识别模块实现	15
3.4.3 抓取姿态模块实现	17
3.4.4 逆运动学模块实现	19
3.4.5 系统集成	19
4 系统仿真结果与分析	22
4.1 系统仿真	22
4.1.1 系统仿真环境	22
4.1.2 系统仿真搭建	22
4.1.3 系统仿真实验设计	24
4.2 系统仿真结果	27
4.3 系统仿真分析	30

5 结论与展望	32
5.1 结论	32
5.2 展望	32
参考文献	33
致谢	35
附录 仿真说明	36

1 引言

1.1 虚拟仿真抓取系统研究背景

随着人工智能与机器人的发展，机器人自主抓取在仓储物流等许多场景开始应用。很多实验依赖与真实硬件平台，实验成本与系统搭建过程的花费都十分的昂贵；此外这类实验对场地和设备配置要求较高，整体灵活性有限，在调试阶段还容易受到环境条件和设备状态的影响，特别是在实验时的数据频繁调整或者进行大规模测试时效率不够理想。

而伴随着技术的发展，虚拟仿真技术逐渐被引入到机器人抓取研究当中。虚拟环境在构建三维环境之后，可以在不依赖真实硬件的情况下完成抓取算法的设计和验证过程。例如（图 1）所示的大型运装抓取机器，实际应用往往在较为复杂的线下工厂，同时设备本身价值较高，一旦进入现场调试阶段，无论是人力投入还是时间成本都会明显增加，而且调试过程本身也不够灵活。如果在操作中出现失误，不仅会带来设备损耗，还会引发安全问题。基于这些现实约束，在虚拟环境中先完成系统验证和性能评估，一方面可以提前发现潜在问题，另一方面也有助于降低实际部署时的不确定性。



（A）工厂货运机器人



（B）工厂抓取机器人

图 1 工厂机器人

1.2 虚拟仿真抓取系统研究目的与意义

本研究针对虚拟仿真环境下的机器人抓取系统展开，通过在虚拟环境里进行实验仿真，从而在无需依赖真实硬件的情况下进行抓取任务的初步验证。

仿真通过设置不同的物体摆放形式，并在实验环境中加入适量干扰因素来验证抓取算法在虚拟环境中的运行效果与稳定性，使实验条件贴近真实应用中可能出现

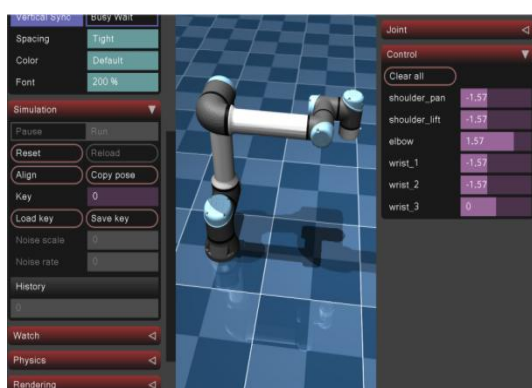
的各类不确定状况，还能进一步分析这类算法在复杂多变场景下的适应能力与鲁棒性。还可以根据具体的实验需求，快速调整系统参数以及切换不同算法，对比其在不同物体布局，抓取姿态及环境条件下的表现，从而对抓取算法的综合性能进行评估。并且虚拟环境能够开展多次测试，有效减少对真实物理设备的依赖与实验过程中的风险与成本，让实验更易重复开展，整体执行效率更高。

2 虚拟仿真抓取系统研究

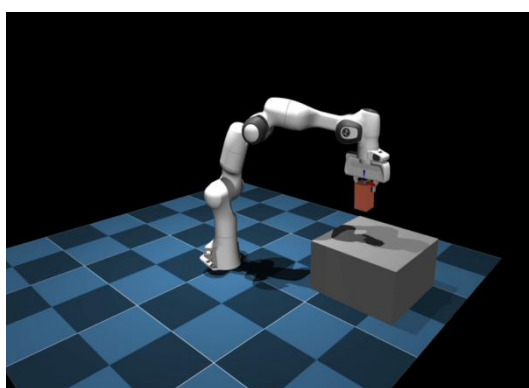
2.1 虚拟仿真抓取系统研究现状

早期研究大多以动力学仿真验证为核心，通过物理建模的方式对机械臂的实际运动过程进行模拟分析。以（图 2）为代表的仿真引擎Mujoco具有稳定的控制求解器与高效的动力学模拟能力，被广泛应用于机器人控制算法与强化学习任务的基准测试^[1]。通过这样的仿真平台，研究者能够降低物理实验中的风险与成本，并在虚拟环境中实现关节控制、轨迹规划及力学反馈分析等简单的测试。

随着抓取目标复杂度的提高，仿真研究逐渐从单纯的动力学求解扩展到系统级的行为验证。例如平台（图 3）Gazebo将PyBullet和ROS生态系统进行了深度的集成，形成了感知、决策与控制形成了闭环验证框架^[2]。研究者能够在这样的高度集成环境中完成传感器仿真、路径规划及控制策略等更复杂的测试，实现算法在系统级别的迭代与评估。之后随着机器学习的发展，虚拟仿真环境逐渐融入深度学习、计算机视觉与强化学习等智能算法，研究重点转向复杂场景下的视觉感知与抓取控制。

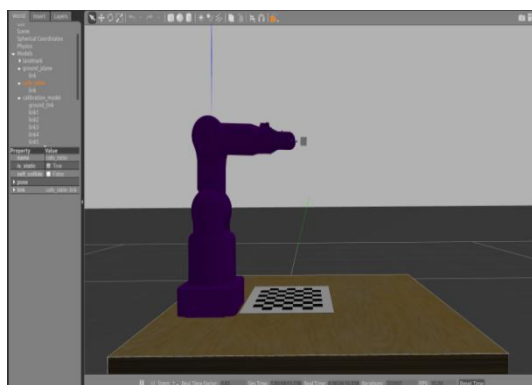


（A）Mujoco平台界面

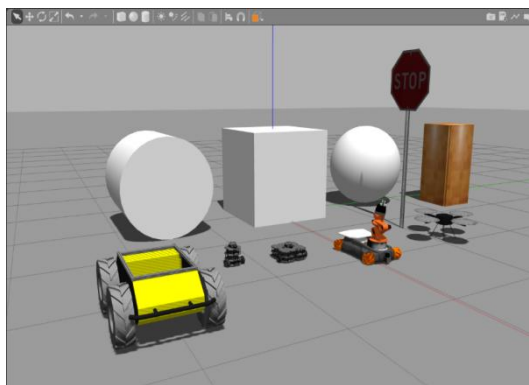


（B）Mujoco平台仿真

图 2 Mujoco平台



（A）Gazebo平台界面



（B）Gazebo平台仿真

图 3 Gazebo平台

2.2 虚拟仿真抓取系统抓取算法研究现状

2.2.1 常见抓取算法方式

目前常见的机器人抓取方式主要可以分为二维与三维抓取^[3]。

二维（2D）抓取（图 4）主要应用于结构相对规则、环境较为固定的工业场景。在该方法中，机械臂通常沿竖直方向从单一视角对目标物体进行抓取。抓取策略主要基于图像平面信息进行规划，抓取表示方式一般采用图像平面内的抓取矩形（或四边形）以及对应的旋转角度来描述抓取位姿。该方法实现简单、计算效率高，适合流水线等标准化作业环境，但由于仅依赖单视角信息所以难以处理复杂堆叠或遮挡严重的场景。

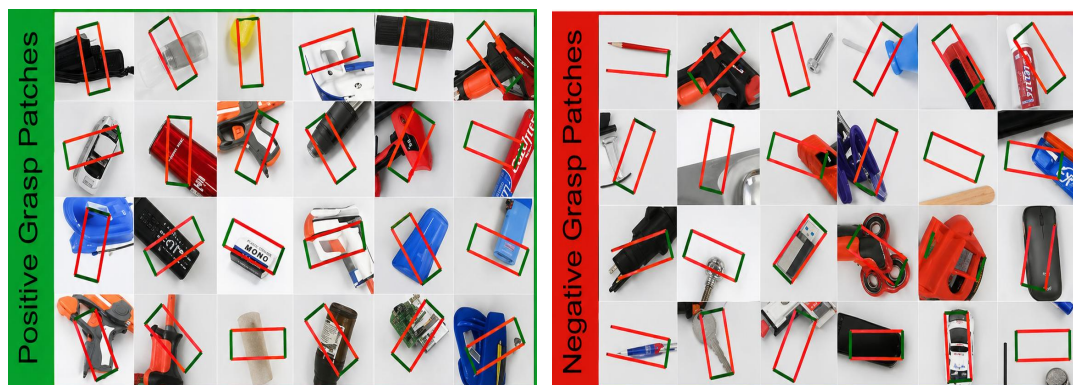


图 4 二维平面抓取

三维抓取中，6-DoF（六自由度）抓取（图 5）能够对空间中任意姿态抓取^[4]，是当前研究的热点方向。这类方法以三维点云为基础，对目标物体的空间位置和姿态进行建模，进而生成相应的抓取位姿。相比于传统方式，6-DoF抓取在灵活性和适应性上更具优势，能够完成复杂场景下多视角、多姿态的抓取任务。而且其模型训练环节可以在虚拟仿真环境中开展，有效减少真实数据采集所需的成本。不过受限于训练数据分布不够全面再加上仿真环境与现实场景存在一定差距，在实际部署时的泛化性能存在一定不足。

形态补全抓取（图 6）方法主要针对点云数据不完整或存在遮挡的情况，通过对目标物体进行形态重建或补全，恢复其潜在几何结构，从而辅助抓取位姿的预测。该方法在处理具有相似结构特征的物体时表现较好，能够在一定程度上弥补感知信息缺失带来的影响。然而由于其依赖于特定类别或先验模型，通用性相对有限，所以通常只适用于特定场景下的抓取预测与评估任务。

2.2.2 抓取算法研究现状

现有抓取研究多依赖二维视觉检测方法实现物体识别与抓取位姿估计。

例如研究员余吉雅等人将YOLOv4-Tiny与RRT-Connect算法结合用于机械臂的自主抓取仿真验证实现了视觉识别与路径规划的一体化设计^[5]。她们的抓取仿真系统能够在虚拟环境中完成目标检测与路径规划任务，但在抓取角度与接触姿态的精确估计方面仍存在不足。同时林邦等人^[6]通过引入深度强化学习优化机械臂视觉抓取控制增强了在动态环境下的抓取性能。之后的研究者，如赵宇晴^[7]则针对6-DOF抓取位姿的估计波动问题提出了向量平均方法以抑制误差。该方法依然依赖点云分辨率和物体几何形状，难以在高复杂场景中保持鲁棒性。针对这些问题，Yin与Li系统性总结了从二维图像到三维空间抓取检测的发展，指出二维视觉方法在高精度三维抓取中的局限性。

研究者为解决上述问题，引入了基于深度学习的三维抓取检测模型（图 7）GraspNet。其中Fang等人^[8]在其提出的GraspNet-1Billion框架下，利用大规模3D点云数据驱动模型学习，从而获取最优的抓取路径与夹爪参数，这种设计实现了2D到3D的闭环控制，赋予了系统应对复杂姿态与多目标动态环境更强的能力。它们的实验也证明该技术在提升抓取精度与系统稳定性方面效果显著。

尽管现有仿真平台在动力学计算、控制验证与数据生成方面取得了显著进展，但是在高真实感视觉渲染抓取任务中的表现仍存在“虚实域差距（Reality Gap）”的问题。所以近年来的研究开始关注高保真渲染、物理一致性建模以及虚实对齐（Sim-to-Real）等方向，试图在保持仿真灵活性的同时提高模型的可迁移性与验证可信度。

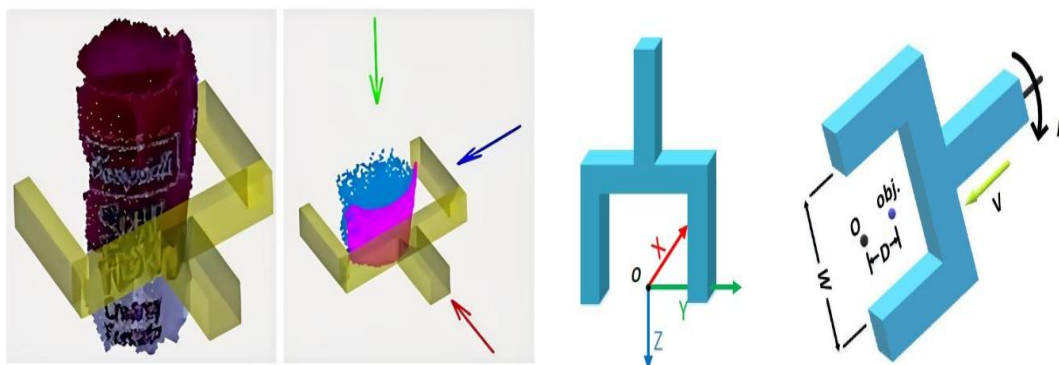


图 5 6-DoF抓取

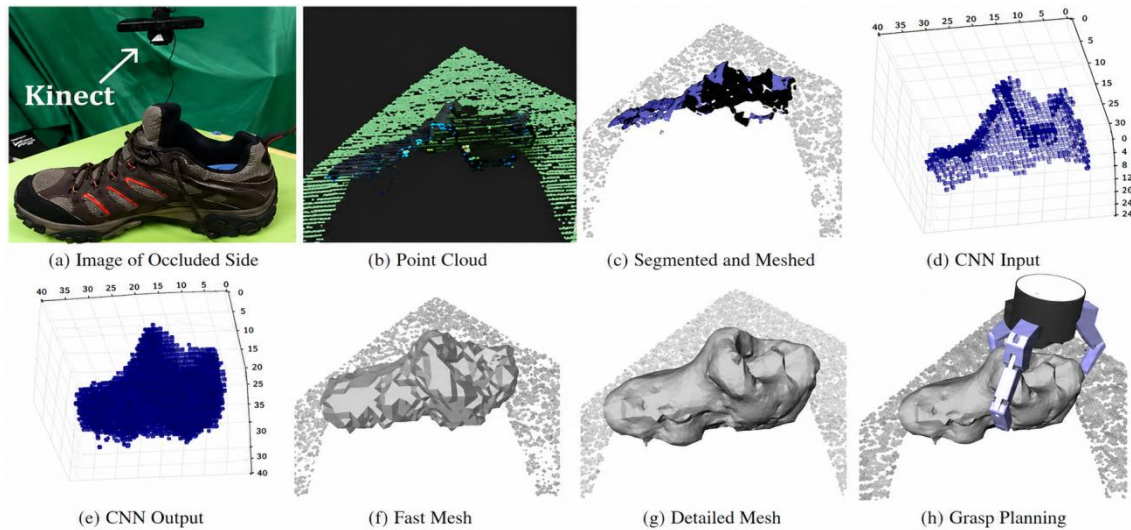


图 6 形态补全抓取

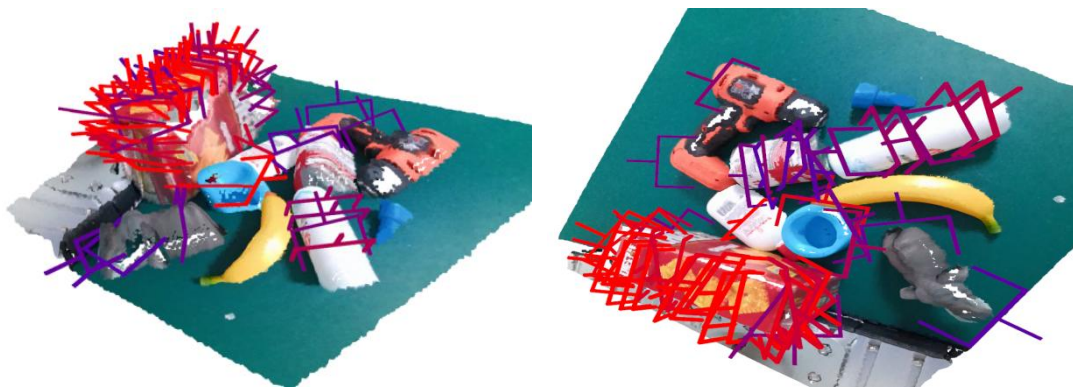


图 7 GraspNet模型抓取

2.3 虚拟仿真抓取系统逆运动学算法研究现状

2.3.1 常用逆运动学方法

逆运动学（Inverse Kinematics, IK）是机械臂控制与运动规划的核心方法。目前主流 IK 算法包括 CCD（Cyclic Coordinate Descent）、FABRIK（Forward and Backward Reaching Inverse Kinematics）以及基于雅可比矩阵（Jacobian-based）迭代方法。这些算法的计算开销低、实现相对简单、收敛速度快，常应用于实时仿真和工程控制中。

一、CCD 算法（图 8）通过逐关节迭代逼近目标位姿，适合多自由度机械臂的快速控制。

策略思路：

（1）从末端开始回溯：算法从骨骼链的倒数第二个关节开始，依次向根节点方

向进行遍历调整。

(2) 计算旋转对齐向量:

当前向量: 从当前关节指向当前末端受动器的向量。

目标向量: 从当前关节指向目标点 (Target) 的向量。

(3) 局部旋转修正: 计算当前向量与目标向量之间的夹角, 并将当前关节旋转该角度, 使得当前向量与目标向量重合。此时, 末端点会由于该旋转而更加靠近目标。

(4) 循环迭代: 上述过程在整条骨骼链上重复进行。如果一轮结束后末端仍未到达目标, 则开启下一轮循环, 直至满足精度要求或达到最大迭代次数。

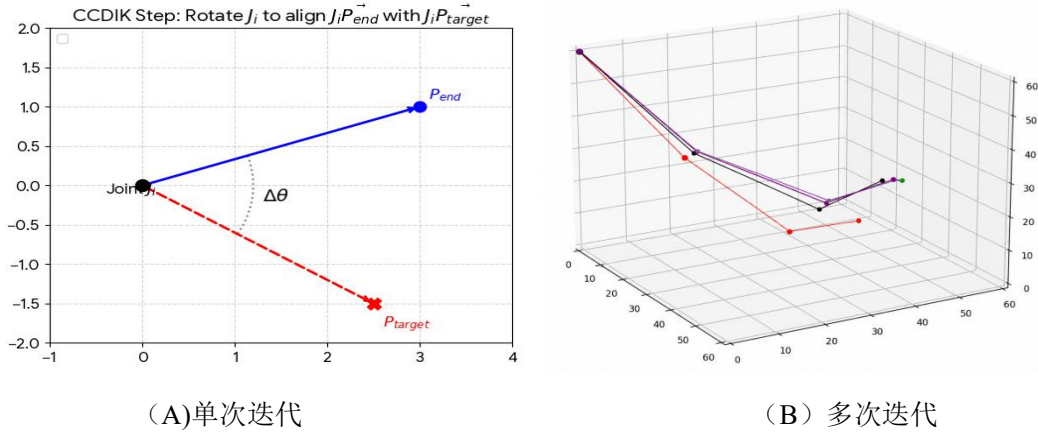


图 8 CCDIK算法

二、FABRIK 算法 (图 9) 利用正向和反向链条迭代更新关节位置, 实现了收敛速度与数值稳定性的平衡, 尤其适用于高自由度机械臂和虚拟仿真动画场景^[9]。

策略思路: FABRIK 的单次迭代分为两个阶段 (前向和后向):

(1) 前向拉伸 (Backward Reach - 从末端到根部): 将末端受动器 (End Effector) 移动到目标位置 (Target)。为了保持骨骼长度不变, 计算前一个关节与当前关节的连线, 沿着这个方向移动前一个关节, 使其与当前关节保持原有的骨骼长度。重复此过程, 直到更新完所有关节 (此时根节点偏离了原始位置)。

(2) 后向修正 (Forward Reach - 从根部到末端): 将根节点 (Root) 移回其原始初始位置。同样为了保持骨骼长度, 依次更新后续关节的位置, 直到更新完末端受动器。

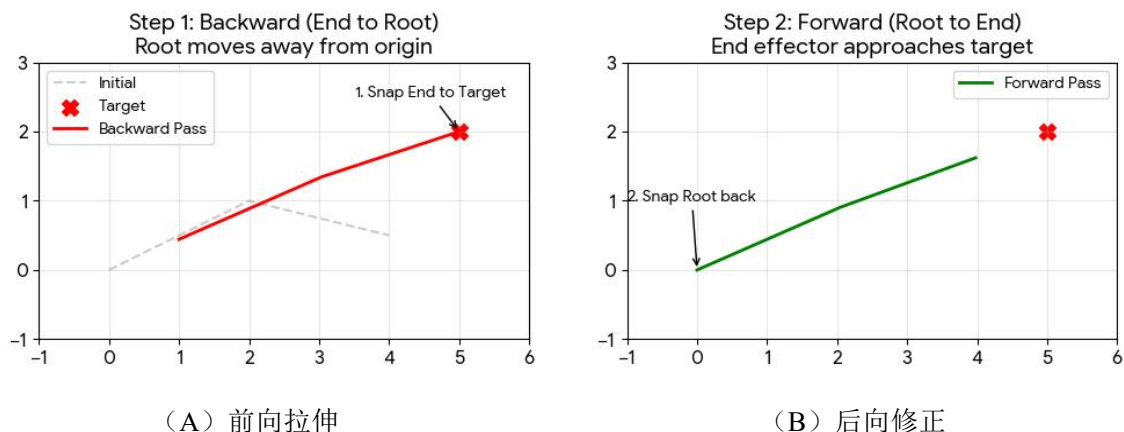


图 9 FABRIK算法

三、基于雅可比矩阵的方法（图 10）通过解析末端位姿与关节速度的线性关系，可有效处理奇异位姿和冗余自由度问题，并能够与约束优化、碰撞检测和力控制方法结合^[10]。策略思路：

（1）构建关系矩阵

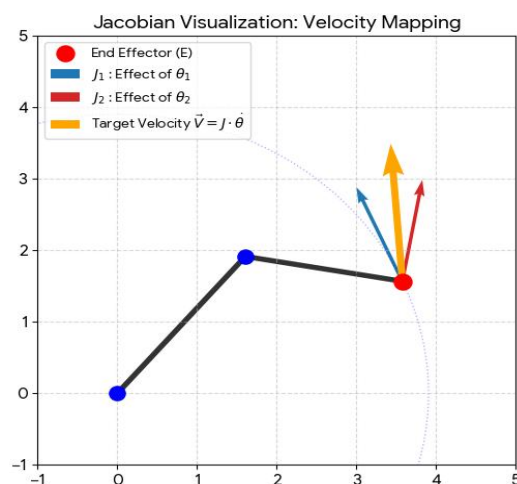
在当前机械臂姿态下，计算每个关节发生微小变化时，末端执行器在空间中的运动方向与变化趋势。将所有关节对末端运动的影响统一整理，形成一个关系矩阵。

（2）求解关节增量

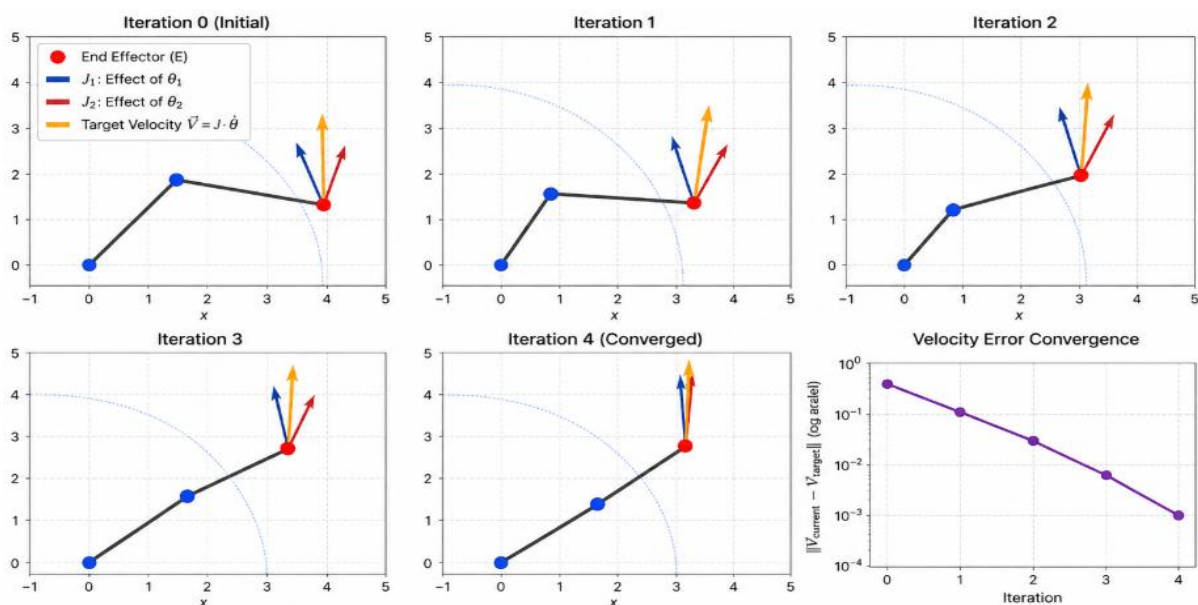
根据目标位置与当前末端位置之间的误差，反向推算需要调整的各个关节变化量。

（3）迭代更新关节状态

将计算得到的关节变化量应用到当前关节上，更新机械臂姿态。随后重新计算末端位置与目标之间的误差，并基于新的姿态重新构建关系矩阵和求解关节变化。该过程不断重复，逐步逼近目标位置。



（A）单次求逆解



(B) 迭代求逆解

图 10 雅可比矩阵算法

2.3.2 Unity中的逆运动学方法

在Unity平台上使用的IK算法是FastIK (CCD)算法，FastIK通过建模的父子节点的层级关系进行关节迭代求解（图 11），算法高度依赖层级结构，节点之间耦合紧密，可移植性不强，限制了它在复杂、多自由度机械臂建模与控制中的应用。因此可以采用节点式通用化设计方法，通过显式输入关节序列代替层级自动检索，让算法能适配不同机械臂结构，同时在求解的稳定性和收敛性能上做出优化^[11]。

```
void Awake()
{
    //initial length
    Bones = new Transform[ChainLength + 1];
    //InitialRotation = new Quaternion[ChainLength + 1];
    TargetInitialRotation = Target.rotation;
    EndInitialRotation = transform.rotation;

    var current = transform;
    CompleteLength = 0;
    for (int i = ChainLength - 1; i >= 0; i--)
    {
        CompleteLength += (current.position - current.parent.position).magnitude;
        Bones[i + 1] = current;
        Bones[i] = current.parent;
        //InitialRotation[i + 1] = current.rotation;
        //InitialRotation[i] = current.parent.rotation;
        current = current.parent;
    }
    if (Bones[0] == null)
        throw new UnityException("The chain value is longer than the ancestor chain!");
}
```

图 11 FastIK关节层级迭代代码

3 虚拟仿真抓取系统实现

3.1 虚拟仿真抓取系统结构分析

现有虚拟仿真的机器人抓取系统（图 12），可以理解为三个部分组成：仿真平台、感知抓取模块、运动控制算法。其中仿真平台给整个系统提供实验环境^[12]，主要用来搭建三维场景，以及生成图像数据。感知抓取模块一般会用到深度学习方法，比如目标检测或点云处理，用来识别场景里的物体，并确定它们的位置和姿态。而运动控制算法是执行层，常结合逆运动学方法计算机械臂到达目标物体的运动轨迹并调用抓取。

综合国内外虚拟系统现状，首先仿真平台与真实物理环境之间存在视觉差距，Mujoco、Gazebo 等主流平台在视觉渲染精度与物理建模保真度上拥有差距^[13]，难以完整还原真实场景中的光照变化、物体材质属性，这会影响视觉模型在仿真中的训练效果，也不利于和真实环境对照验证。本文采取画面更加真实的引擎Unity来进行实验。

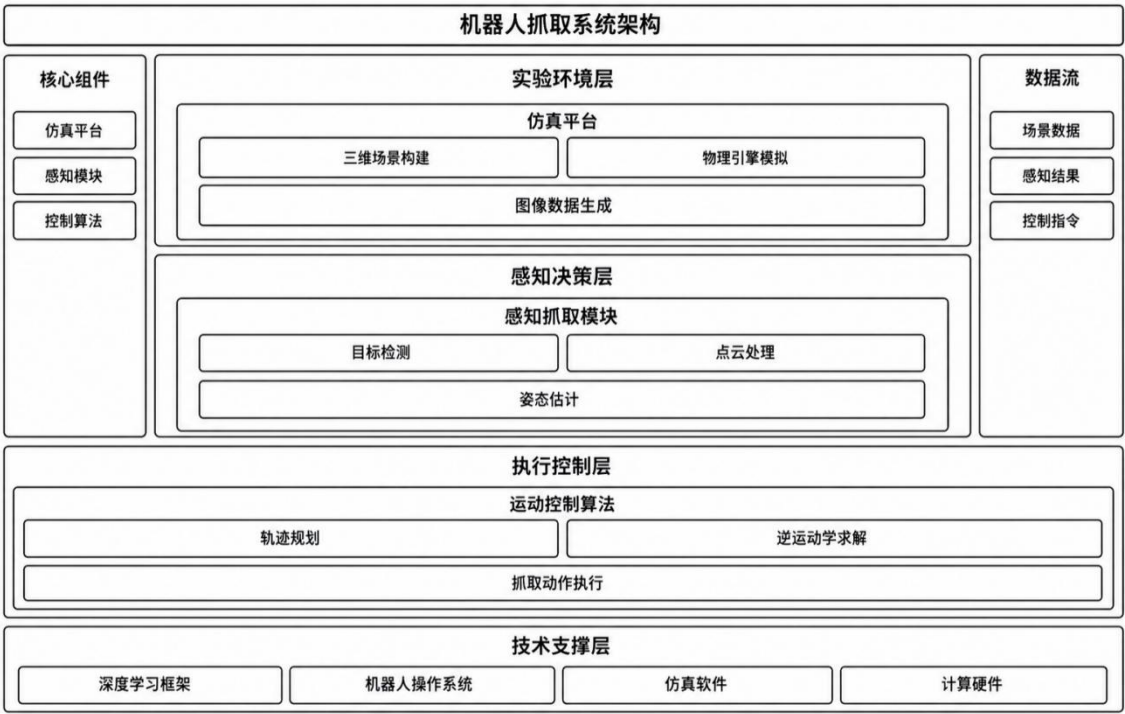


图 12 虚拟仿真抓取系统结构

3.2 虚拟仿真抓取系统研究路线

（1）环境搭建

机械臂建模是系统实现的基础，依据实验需求构建多自由度模型，定义关节参

数及运动约束，并引入碰撞检测与接触机制，以提升仿真物理一致性。

(2) 目标定位

在环境感知层，采用深度学习实现目标识别与跟踪，并对其位姿进行实时估计，为抓取决策提供输入。

(3) 姿态生成

基于抓取生成模型生成候选抓取姿态，并通过评价函数筛选最优方案，以提高抓取成功率与鲁棒性。

(4) 运动控制

利用逆运动学求解关节参数并完成路径规划，在满足约束条件下优化路径，同时结合反馈进行闭环控制，以提高执行精度与稳定性。

(5) 系统验证

集成模块，并搭建虚拟仿真平台，实现系统闭环运行。之后对识别精度、抓取成功率及运动性能进行评估，分析系统可靠性。

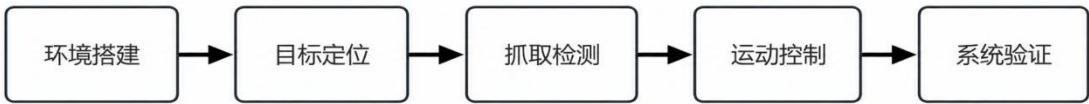


图 13 研究路线图

3.3 虚拟仿真抓取系统研究方法

在这项研究里，机械臂模型的搭建需要紧密结合实验需求来整体设计，借助Blender等建模工具，把机械臂的关节、连杆和末端执行器等部分细化搭建，把各个结构关系先理清楚。在这个过程中，每个关节的转动方式、可活动范围以及整体尺寸都需要进行设定，以免后续运动时出现不合理情况。建模时不仅要关注结构是否完整，还要考虑机械臂实际使用时的表现，比如抓取范围，以及面对不同大小、不同形状的物体能否正常抓取。模型建立完成后，还需导入虚拟仿真平台，结合物理引擎并加上一些运动限制，比如关约束和碰撞检测。

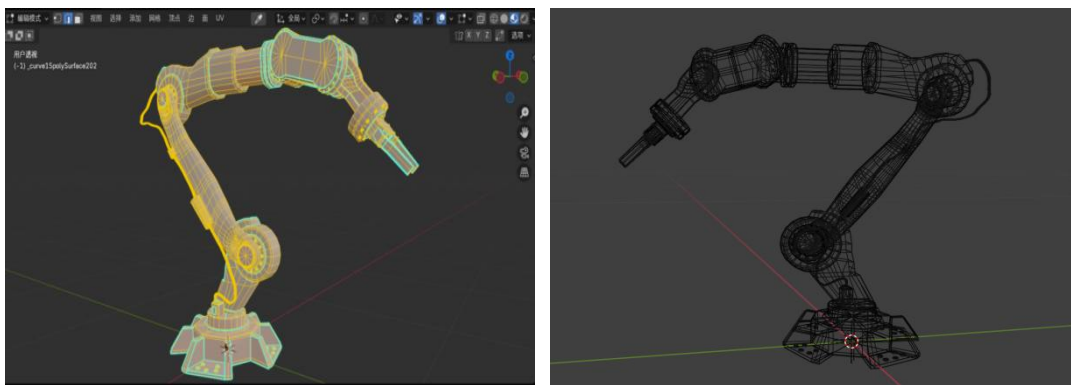


图 14 Blender机械臂模型

在Unity虚拟环境中，为构建基础视觉感知模块，需要对虚拟摄像机进行参数化配置，以逼近真实相机的成像模型（包括内参、视场角及分辨率等），并以设定的固定帧率对场景进行连续渲染与采样，从而获取时序一致的图像数据流。基于该机制，可生成连续的RGB图像序列，作为后续视觉感知与目标识别任务的基础输入。

（图 15，图 16）YOLO模型会对画面进行分析，识别目标物体并同时给出一些关键信息，如物体的类别、物体的位置，以及判断的置信度^[14]。在模型推理结果的后处理阶段，引入置信度阈值（**confidence threshold**）对检测结果进行筛选，剔除低置信度候选框，仅保留具有较高可靠性的目标预测，从而有效降低误检率并提升检测结果的稳定性。在此基础上，系统可在Unity虚拟环境中实现对多类别目标的实时识别与空间定位，并获取对应的位置信息。

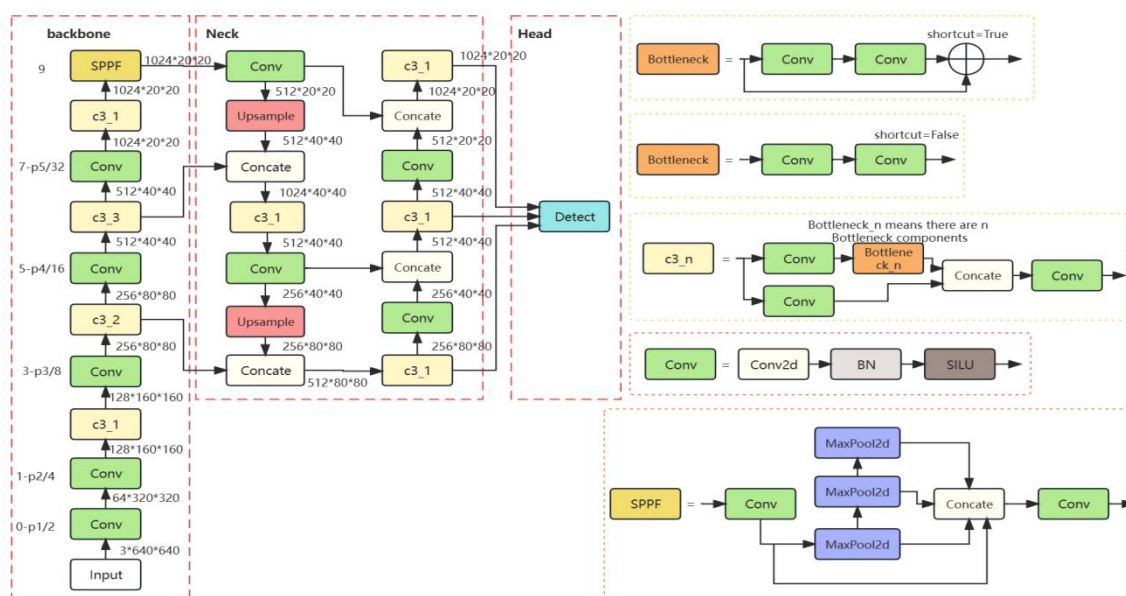


图 15 YOLO模型网络图^[15]

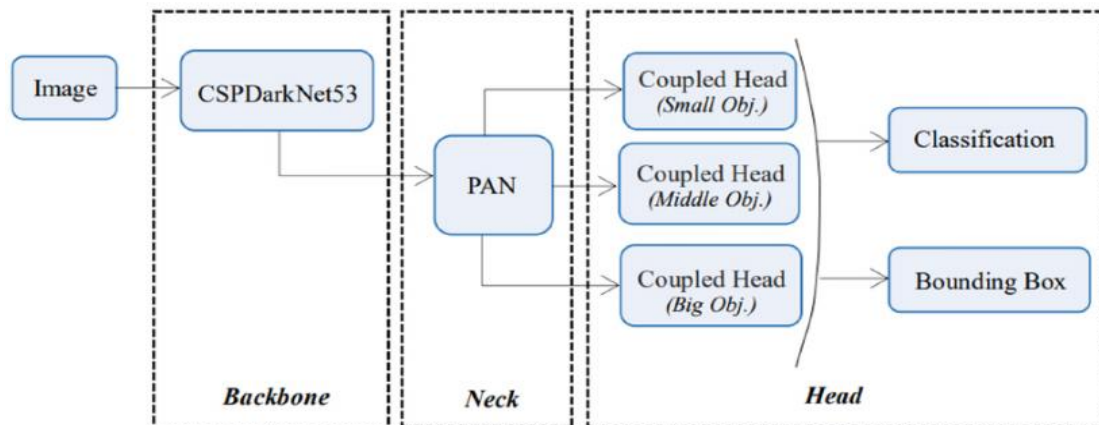


图 16 YOLO模型架构图^[16]

识别目标物体后，采用GraspNet预训练模型^[17]（图 17）对目标物体进行抓取候选姿态预测。首先GraspNet模型将获取摄像机中的RGB-D图像生成三维点云数据。模型会根据当前物体的形状和空间分布，生成多个不同的抓取姿态，每一个方案里中包含关键信息，具体的抓取位置、夹爪方向，以及夹爪的宽度和角度还有置信度。后续抓取决策的时候筛选将筛选出一个最优的姿态执行。

为了从多个候选结果中选取最优抓取姿态，需要对生成的抓取结果进行筛选与排序。具体方法基于模型输出的置信度评分进行阈值过滤，以及结合机械臂运动学约束对抓取可达性进行判断^[18]。通过综合考虑抓取稳定性与执行可行性，最终确定最优抓取姿态，并将其作为机械臂执行抓取操作的输入参数。

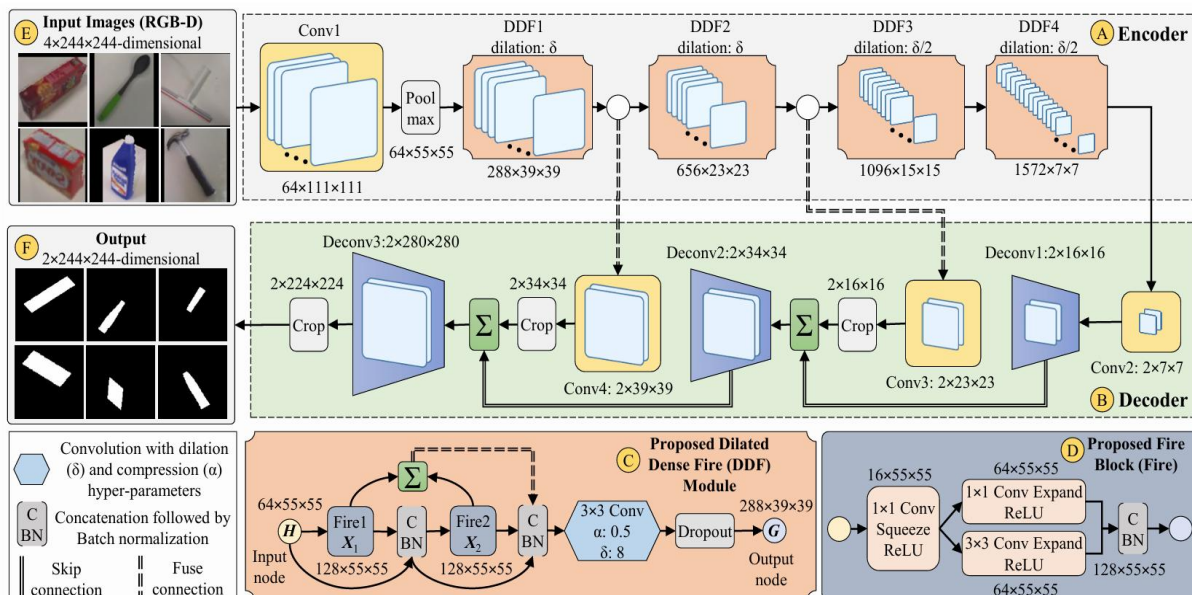


图 17 GraspNet模型结构图^[17]

获取到模型推理输出后，其结果通常不能直接用于Unity中的运动控制模块，需

要首先进行坐标系转换。具体而言，目标检测与位姿估计算法输出的三维位置与姿态信息一般定义在虚拟相机坐标系下，而机械臂的运动规划与控制则基于世界坐标系或机器人基坐标系进行描述。因此，有必要通过外参对位姿进行变换，将其从相机坐标系映射至统一的参考坐标系中。

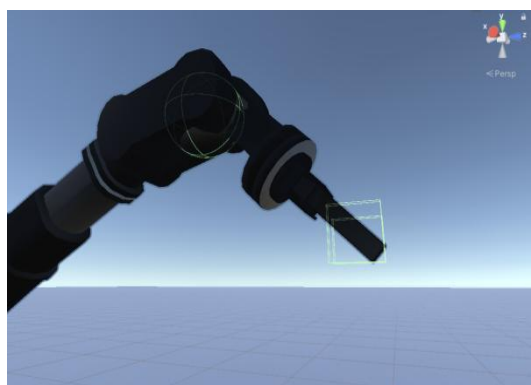
3.4 虚拟仿真抓取系统设计方案

3.4.1 机械臂的模型实现

在Blender中建立机械臂的骨骼模型，依据实际机械臂的结构参数对各关节结构标注其旋转轴方向、自由度类型以及对应的运动范围。并对关节之间的连接关系与层级结构进行合理设置。



(A) 导入Unity的机械臂



(B) 机械臂添加碰撞体

图 18 机械臂

在此基础上，对每个关节的初始角度及其旋转限制进行参数化处理，通过设定关节角度上下限约束（图 19），确保机械臂在运动过程中始终满足物理结构与运动学约束条件。使得机械臂在执行抓取任务时避免出现关节越界或非物理运动等问题。

```
void InitJointLimits()
{
    limits = new JointLimits[5];

    limits[0] = new JointLimits { allowY = true, y = new Vector2(-180, 180) };
    limits[1] = new JointLimits { allowX = true, x = new Vector2(-60, 100) };
    limits[2] = new JointLimits { allowX = true, x = new Vector2(-210, 30) };
    limits[3] = new JointLimits { allowX = true, x = new Vector2(-90, 90) };
    limits[4] = new JointLimits { allowX = true, x = new Vector2(30, 90) };
}
```

图 19 机械臂转动约束

3.4.2 目标识别模块实现

在本系统目标识别模块中，通过在仿真环境搭建的摄像机组件对目标的RGB图像数据^[19]进行获取。并将该数据作为YOLO模型的输入。

YOLO模型的训练过程：通过在Unity中搭建虚拟场景并调节光照强度、物体数量以及摆放位置等参数模拟不同环境下的抓取场景，采集得到500张原始数据图像。之后使用Python对数据集进行进一步扩充与增强处理，包括图像亮度与对比度调整等操作。通过以上方法生成的数据样本，总计1500张。然后对数据进行目标标注，生成对应的标签文件，并按照训练集、验证集划分数据集。随后在Python环境下配置深度学习训练框架，加载预训练权重作为模型初始参数。训练过程中，设置合适的学习率以及训练轮数，并通过持续监测损失函数等指标来评估模型性能。在训练完成后，选取性能最优的权重文件作为最终模型，模型部署到系统中实现对目标物体的实时检测与定位。

在目标识别过程中，使用YOLO模型对目标进行识别并获取信息，之后对信息进行处理，筛选得到信度较高的目标，并对每个类别选择具有最高置信度的边界框。最后将提取的位置信息进行格式转换处理后，通过接口调用的方式传递至后续的Unity接口。

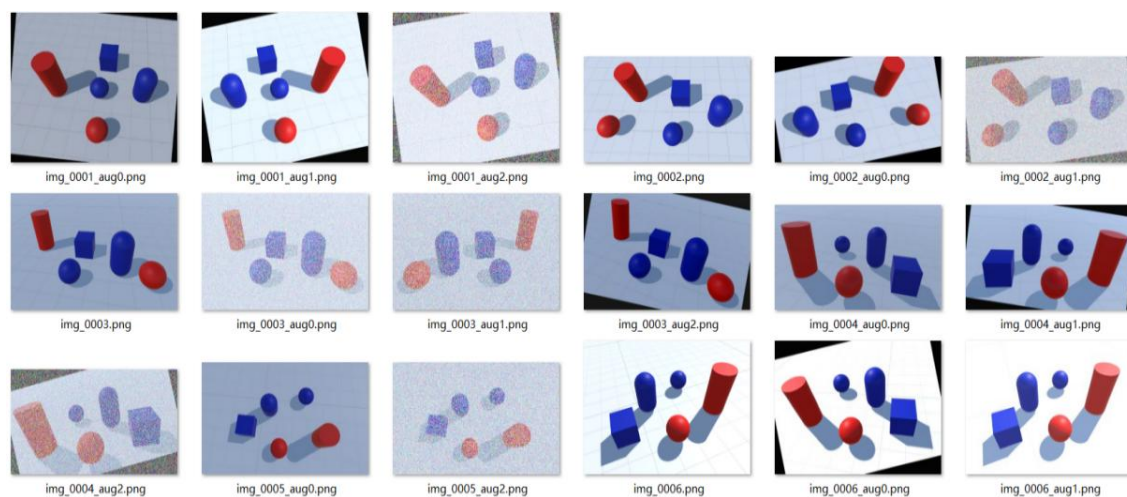


图 20 YOLO训练数据增强图

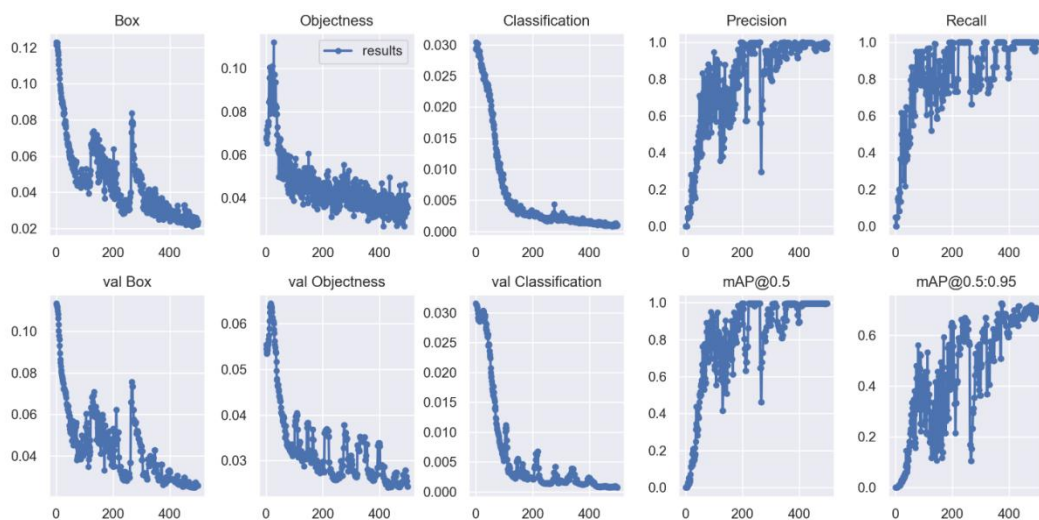


图 21 YOLO模型训练结果图

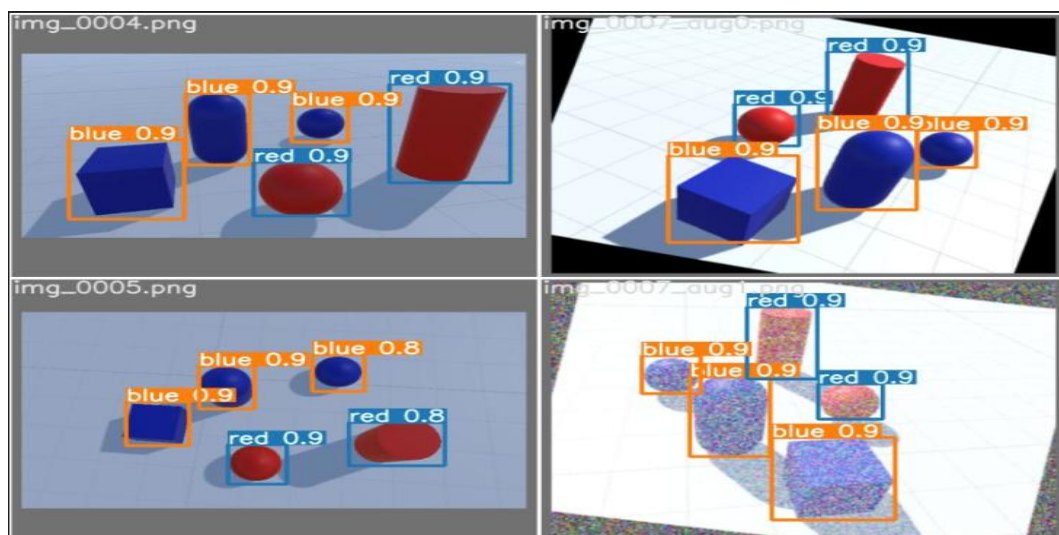


图 22 YOLO训练测试结果图

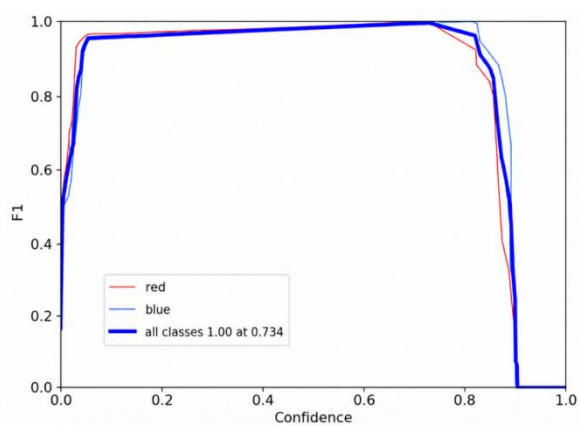


图 23 YOLO训练评估曲线图

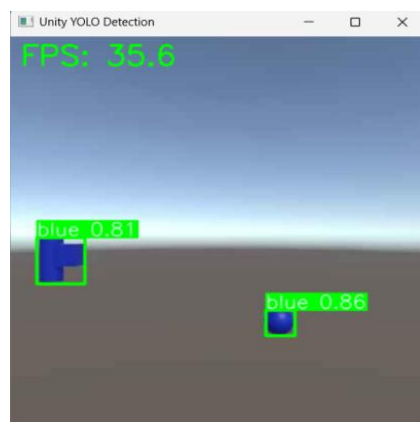


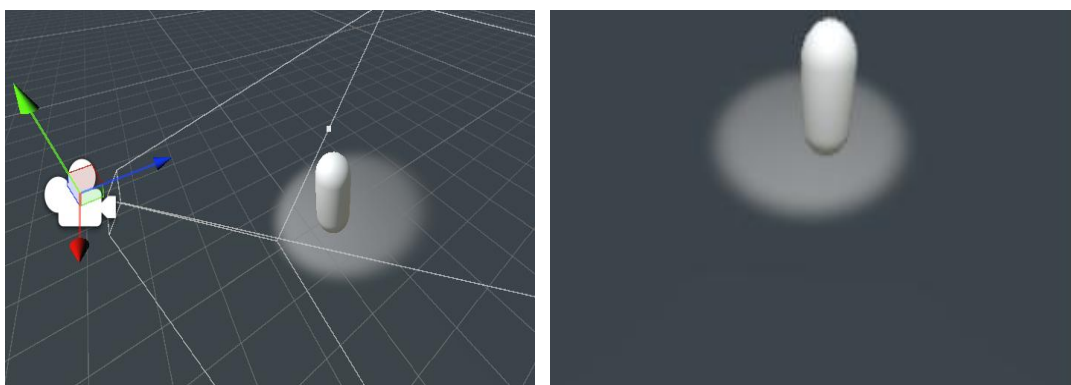
图 24 YOLO视频流检测图

3.4.3 抓取姿态模块实现

在目标识别的同时生成抓取姿态。该阶段以虚拟摄像机获取的物体深度图信息（图 25）为输入再结合目标物体的空间特征辅助（图27，图 28）生成可行的抓取姿态。

GraspNet的训练过程：本系统并未采用传统方式对数据集进行人工采样与标注，而是采用了一种基于模型自生成抓取数据的反向训练方法。首先将场景中的物体的深度图与点云数据输入到初始GraspNet模型中，由模型自动对目标物体进行抓取采样生成多个候选抓取姿态。这些抓取姿态包含机械臂末端执行器的抓取方向、旋转角度以及夹爪宽度等信息。随后对模型生成的抓取结果进行筛选与评估，筛选过程中结合抓取姿态是否发生机械臂碰撞，抓取方向是否满足夹爪闭合等条件，筛选出抓取成功率较高的有效抓取姿态。在获得抓取结果后，将这些由模型生成并筛选后的抓取姿态重新作为训练数据对自身进行迭代优化，从而逐步提升模型对抓取姿态的生成能力。本系统共进行了达100次的迭代训练。在训练过程中，模型不断利用新生成的高质量抓取数据更新网络参数，使抓取姿态预测结果逐渐收敛。

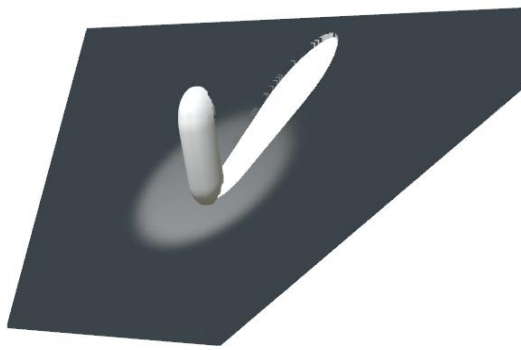
在抓取姿态生成过程中，引入GraspNet模型对目标物体进行处理，生成多个候选抓取姿态（图 28）。每个候选姿态包含抓取位置、抓取方向以及夹爪开合宽度等关键参数用于描述机械臂末端执行器的抓取方式。之后系统会按既定的标准设置阈值对比姿态信息做一轮筛选和简单优化，挑出其中合适的方案作为最终执行指令。最后再把这个筛出来的姿态传给运动控制模块。



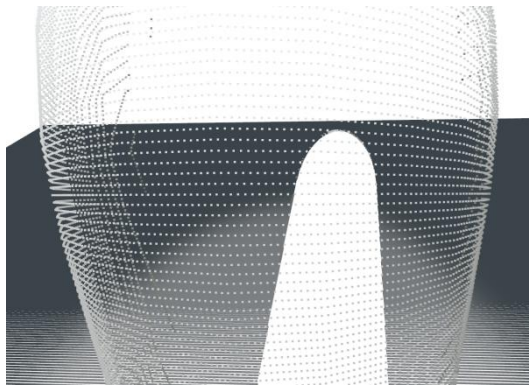
（A）Unity场景放置摄像机与物体

（B）Unity摄像机中的物体

图 25 Unity环境图



(A) 点云图全景



(B) 点云图近景

图26 点云生成结果图

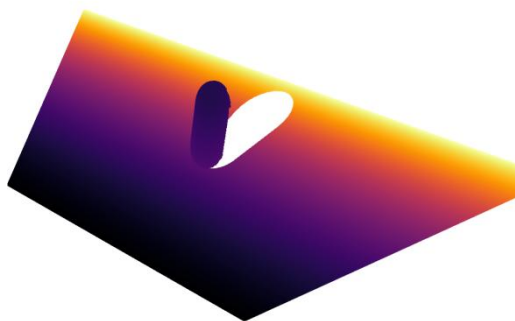


图 27 深度图辅助成像结果

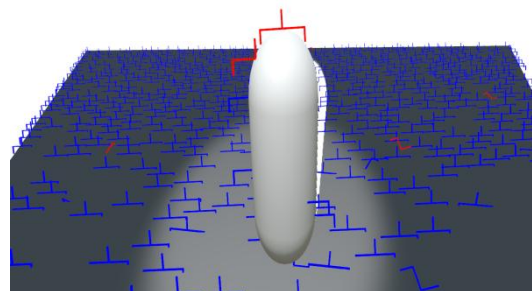


图 28 GrasNet抓取姿态测试结果图

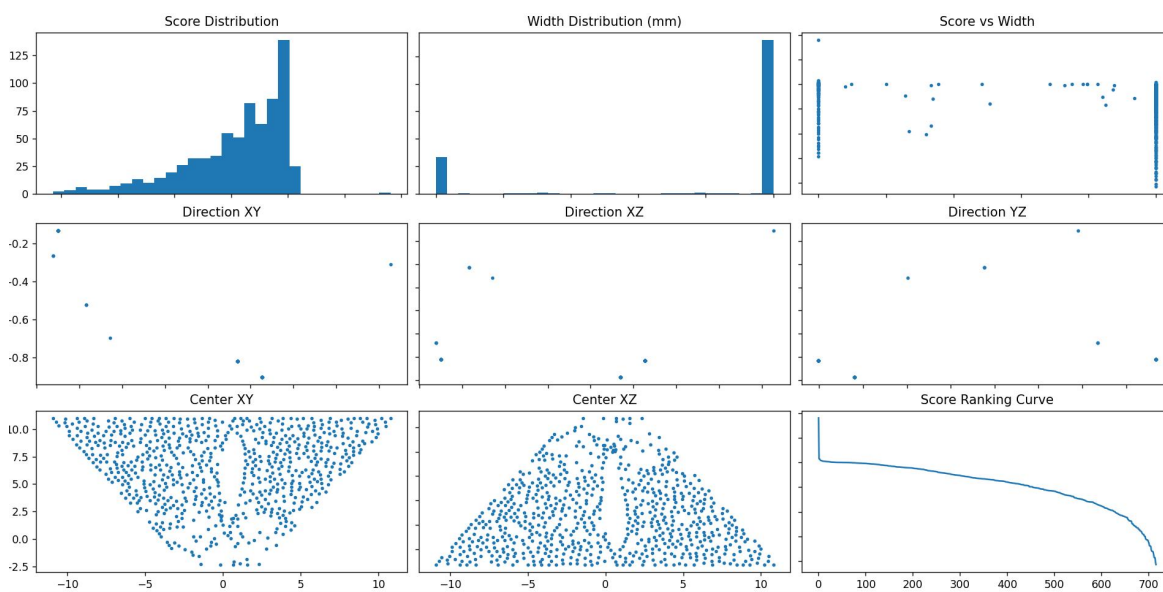


图 29 GrasNet测试结果图

3.4.4 逆运动学模块实现

在机械臂运动控制过程中，采用显式关节建模的方法（图 30，图 31），对机械臂各关节进行单独设定控制。将机械臂表示为由多个刚性关节组成的串联结构，从而实现对整体运动的精细化控制。

在具体实现过程中，通过绑定显式机械臂节点（图 32），再接收前两个模块分别传送的目标位置信息与抓取姿态信息来实现从基座到末端的运动传递，并对关节角度进行逐级计算与迭代调整，并结合预先设定的关节角度约束与运动范围限制确保各关节运动始终满足机械结构的物理约束条件，避免出现不合理姿态或关节越界现象。



图 30 机械臂模型层级

```
[System.Serializable]
class JointLimits
{
    public Vector2 x = new Vector2(-180, 180);
    public Vector2 y = new Vector2(-180, 180);
    public Vector2 z = new Vector2(-180, 180);
    public bool allowX = false;
    public bool allowY = false;
    public bool allowZ = false;
}
```

图 31 显式输入代码



图 32 显式结构绑定

3.4.5 系统集成

在系统集成过程中，将（图 33）视觉检测模块（YOLO后端接口，连接Unity视频流，并向Unity传输位置信息）与（图 34）抓取姿态模块（GraspNet后端接口，处理深度图数据，并向Unity传输姿态信息）通过前后端接口在Unity平台中进行统一整合，实现各功能模块之间的数据交互与协同工作。具体过程为将视觉检测模块输出

的目标物体位置及类别信息与姿态模块同时生成的候选抓取位姿，经过格式转换与数据封装后，传递至Unity的运动模块接口（图 35）。

数据处理转换：一般三维世界坐标系为右手系，但在Unity中，世界坐标系为左手系。

（1）YOLO检测框数据转换：

$$u = \frac{x_{\min} + x_{\max}}{2}, v = \frac{y_{\min} + y_{\max}}{2} \quad (1)$$

(u, v)为检测框中心点坐标，用于结合深度信息估计目标在三维空间中的位置。

($x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$) 分别为检测框左上角与右下角坐标

（2）YOLO转换Unity坐标系：

$$x = \frac{2 \times u - w}{2 \times p}, z = \frac{h - 2 \times v}{2 \times p} \quad (2)$$

(x, z)为转换到Unity后的坐标轴，(w, h, p)为摄像机的图像内参，w, h代表宽高，p为缩放因子。

```
if __name__ == "__main__":
    threading.Thread(target=video_server, daemon=True).start()
    #视频流传输
    threading.Thread(target=result_server, daemon=True).start()
    #信息返回
    print("Python servers running...")
    while True:
        time.sleep(1)
```

图 33 YOLO模型后端接口

```
if __name__ == '__main__':
    data_dir = 'E:/graspnet_input'
    # 先计算抓取
    gg, cloud = compute_grasps_once(data_dir)
    # 再传输给 Unity
    send_grasps_tcp(gg, host='127.0.0.1', port=5005)
```

图 34 GraspNet模型后端接口

```

// 获取最后一个框
BoxData lastBox = yoloReceiver.latestBoxes[yoloReceiver.latestBoxes.Length - 1];

// 框中心像素坐标
float boxCenterX = (lastBox.box[0] + lastBox.box[2]) / 2f;
float boxCenterZ = (lastBox.box[1] + lastBox.box[3]) / 2f;

// 转换到世界坐标
targetX = (2f * boxCenterX - cameraSender.width) / (2f * pixelToWorldRatio);
targetZ = (cameraSender.height - 2f * boxCenterZ) / (2f * pixelToWorldRatio);

```

图 35 YOLO数据处理

在机械臂运动控制中,将YOLO输出的位置作为逆运动学值输入到机械臂各关节(图 36),让机械臂从当前位置移动到目标位置。再结合抓取姿态模块(图 37)给出的姿态信息控制末端夹子的位置、方向和角度实现抓取,从而完成系统的集成。

```

byte[] lengthBytes = new byte[4];
int readlen = stream.Read(lengthBytes, 0, 4);
if (readlen < 4) return; // 数据不完整,跳过
int length = BitConverter.ToInt32(lengthBytes, 0);
// 读取消息内容
byte[] buffer = new byte[length];
int received = 0;
while (received < length)
{
    received += stream.Read(buffer, received, length - received);
}
string receivedData = Encoding.UTF8.GetString(buffer);
latestBoxes = JsonHelper.FromJson<BoxData>(receivedData); // 解析 JSON 并直接替换 latestBoxes

```

图 36 Unity接收YOLO数据

```

Grasp[] grasps = JsonHelper2.FromJson<Grasp>(msg);

if (grasps != null && grasps.Length > 0)
{
    Grasp g = grasps[0];
    Vector3 dir = new Vector3(g.approach[0], g.approach[1], g.approach[2]);

    if (ik != null)
        ik.SetGraspData(dir);
    return;
}

```

图 37 Unity接收GraspNet数据

4 系统仿真结果与分析

4.1 系统仿真

4.1.1 系统仿真环境

本系统运行于虚拟仿真环境，主要硬件与软件环境如下：

CPU: AMD Ryzen 7 5800

GPU: NVIDIA RTX 3060

内存: 40 GB

存储空间: 2 TB

Unity 版本: Unity 2022.3.5f1

Python 环境: python 3.8

CUDA Version:: 13.1

4.1.2 系统仿真搭建

验证系统首先需要搭建一个基础的虚拟仿真环境（图 39）。并对仿真环境进行空间布局安排、光照的调节、物体的分布等处理，使得在一定程度上贴近真实抓取场景。

环境搭建过程中，首先创建包含地面、工作台及背景的基础场景，用于模拟实际工业或实验环境中的操作空间。在此基础上引入多种目标物体模型，并对其尺寸、形状及颜色进行多样化设置，以提高实验的代表性。同时根据实验需求构建的测试场景，在场景中合理布置虚拟相机去采集RGB图像以及对应的深度信息，作为后续目标检测和抓取姿态估计的输入数据。

在光照设置方面通过调整光源强度与方向（图 40），能够分别设置正常光照和相对较弱的光照环境。不同光照下模型表现差异会比较明显，可以用来模拟现实中不同照明条件对视觉感知带来的影响。

最后对整个仿真环境做一次全面的调试和验证，包括场景能否正确加载、数据采集是否顺畅，还要检查目标检测接口和抓取控制流程之间的连通性，确保各个功能模块在同一个环境下能够协同工作，不会出现断链或异常。

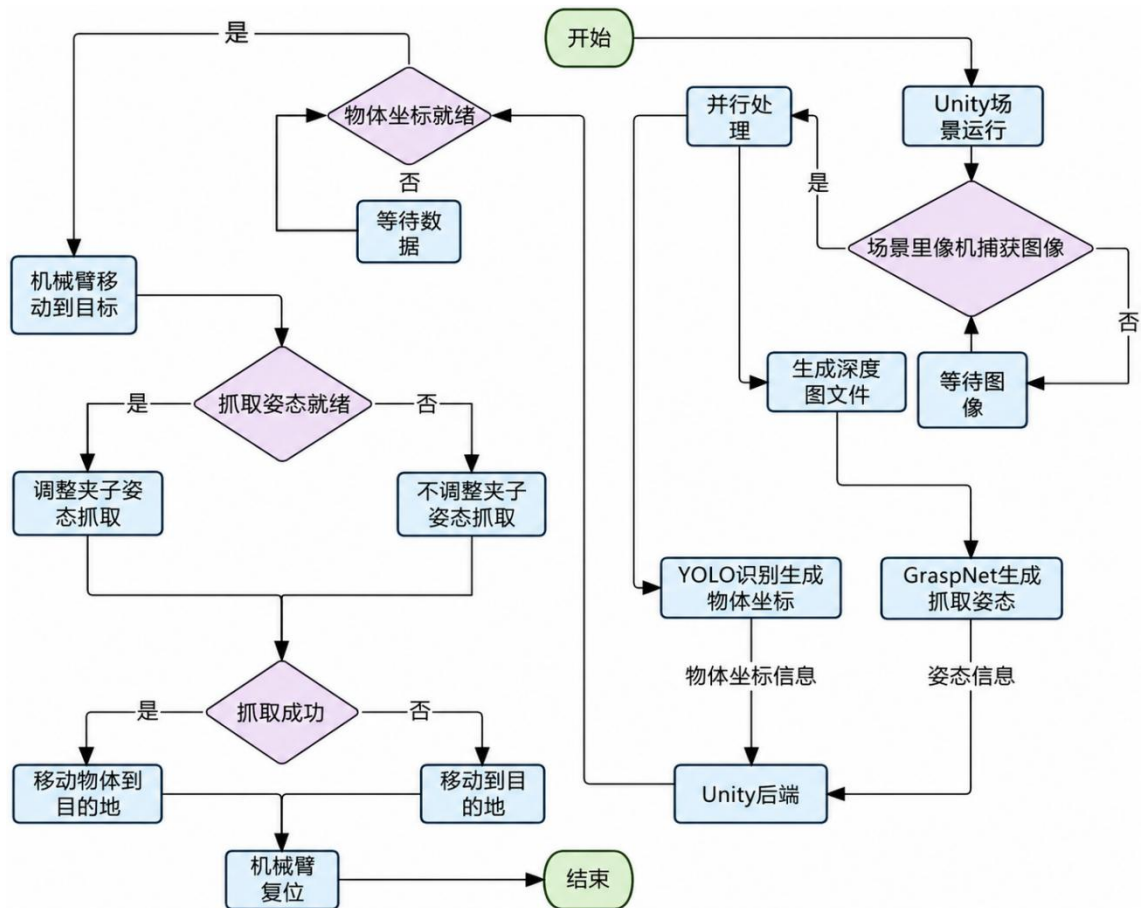
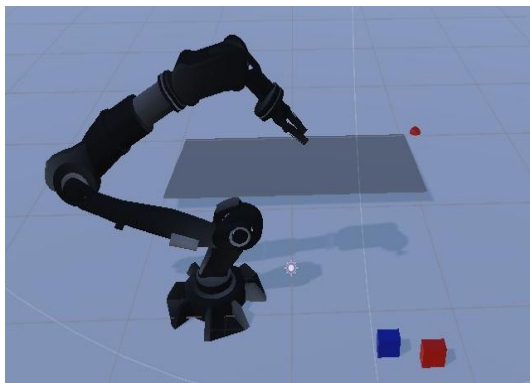
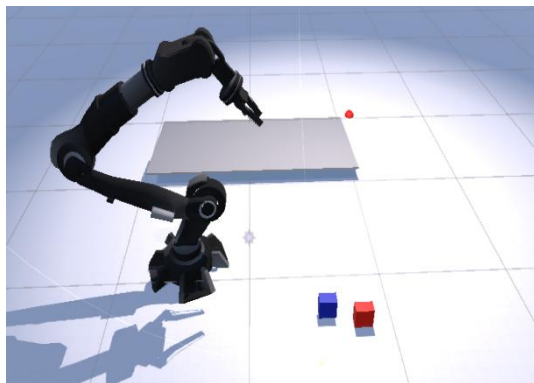


图 38 系统流程图



(A) 调节光照前



(B) 调节光照后

图 39 仿真环境

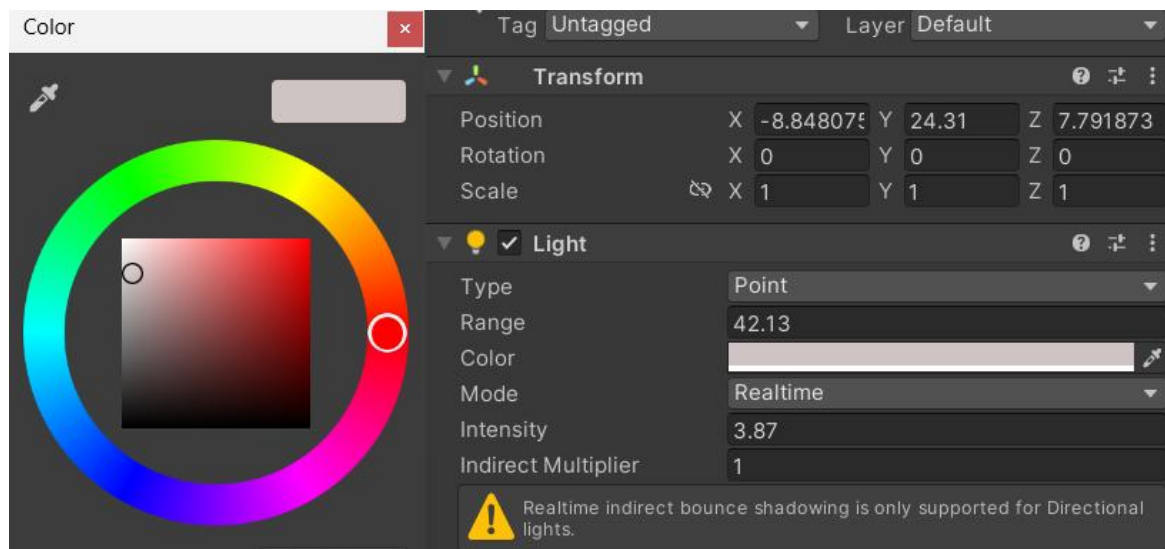


图 40 调节环境光

4.1.3 系统仿真实验设计

实验一：YOLO目标检测性能实验

实验设置，在虚拟仿真环境中采集视频流数据，构建不同检测场景。设置变量如下：

- (1) 目标数量：1个、5个 。
- (2) 光照条件：正常光照、弱光环境 。
- (3) 摆放方式：规则摆放、不规则摆放 。

在各实验条件下，驱动机械臂完成抓取任务，对每种组合条件进行目标检测，记录检测过程视频流数据。

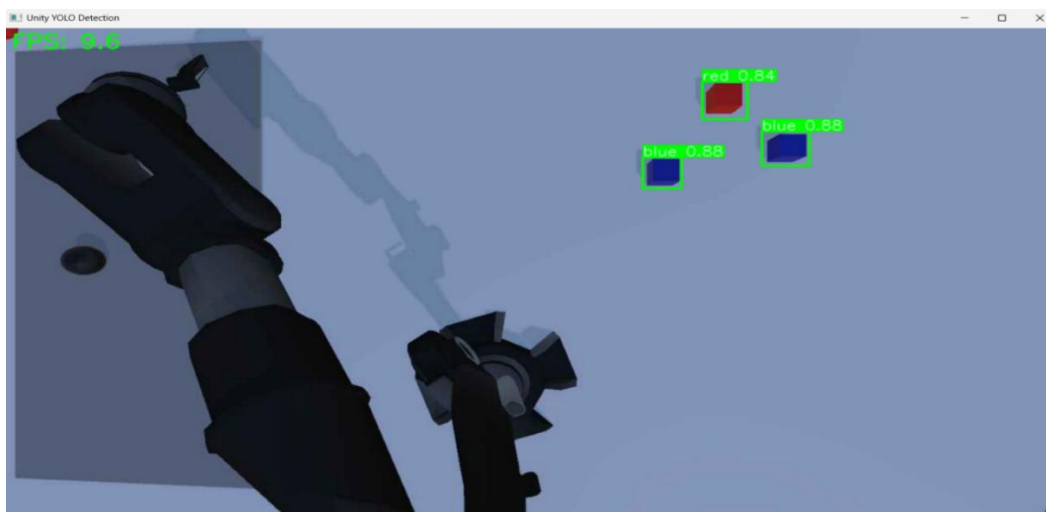


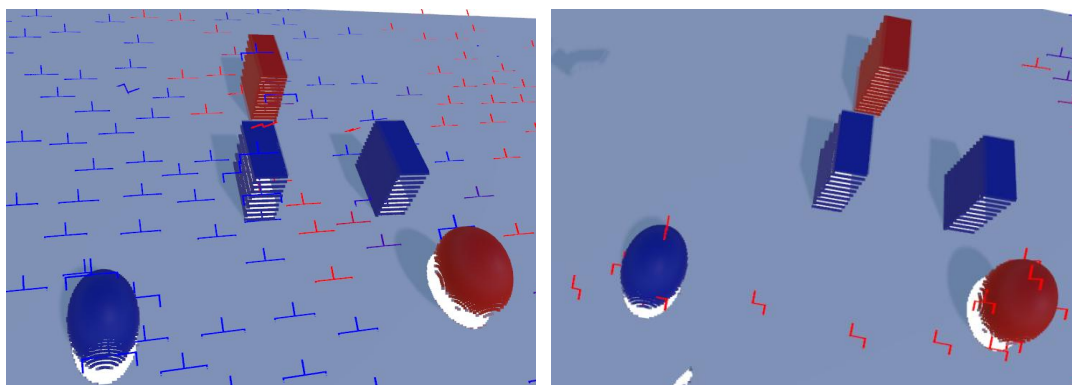
图 41 实验YOLO视频流

实验二：GraspNet抓取姿态生成实验

实验设置，在虚拟环境中搭建物体，对每种组合进行姿态生成实验。每种场景重复实验20次，获取统计结果。设置变量如下：

- (1) 目标数量：1个、5个
- (2) 光照条件：正常光照、弱光环境
- (3) 摆放方式：规则摆放、不规则摆放

在各实验条件下，驱动机械臂完成抓取任务，对每次实验生成的抓取候选进行筛选，并统计全部数量，候选数量及有效抓取数量。



(A) 筛选前的抓取姿态

(B) 筛选后的抓取姿态

图 42 实验GraspNet生成的抓取姿态

实验三：抓取成功率对比实验

实验设置，在虚拟环境中搭建物体，设计对比实验，分析是否使用抓取姿态生成模块对抓取效果的影响。对每种场景分别进行两组实验：不启用抓取姿态生成模块，启用抓取姿态，每种条件下重复进行100次抓取操作。变量设置如下：

- (1) 目标数量：1个、5个。
- (2) 光照条件：正常光照、弱光环境。
- (3) 摆放方式：规则摆放、不规则摆放。

在各实验条件下，驱动机械臂完成抓取任务，记录每组实验的抓取成功情况。

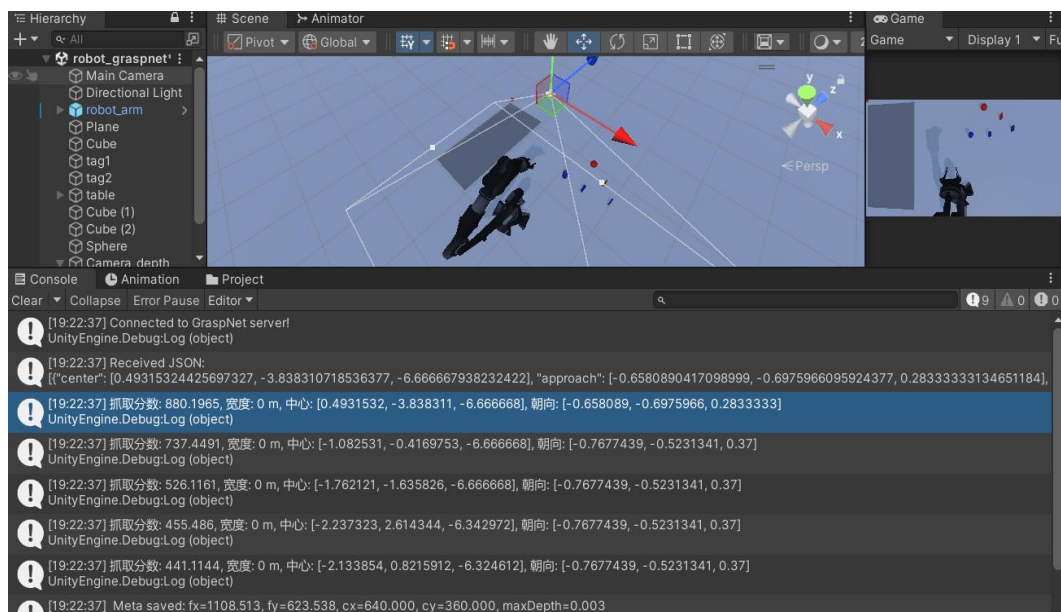


图 43 Unity获取到姿态信息

实验四：机械臂运动控制实验

实验设置，在抓取任务执行过程中，分析机械臂在不同目标分布条件下的运动表现。设置实验变量如下：

- (1) 目标数量：1个、5个
- (2) 目标距离分布：近距离、远距离及混合分布

在各实验条件下，驱动机械臂完成抓取任务，记录机械臂是否成功到达目标位置及运动过程表现。

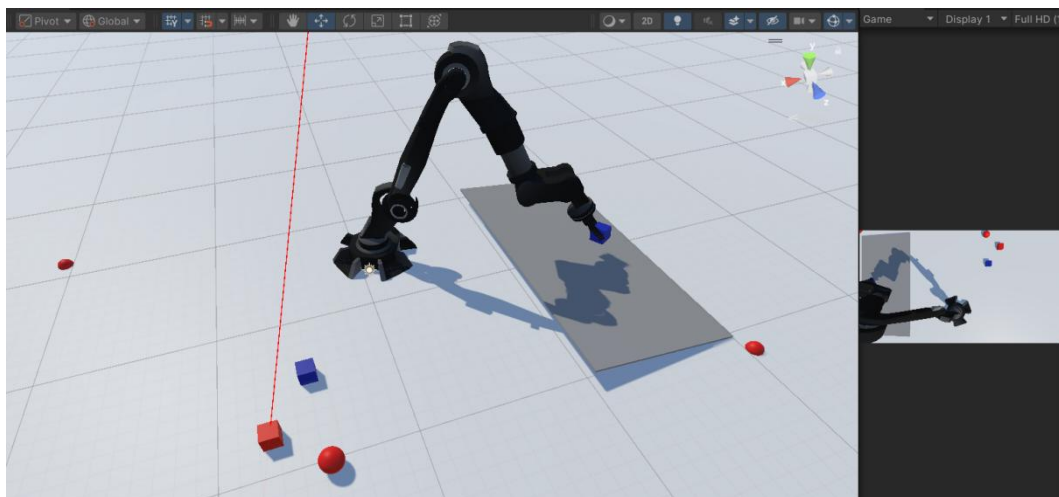


图 44 机械臂抓取物体

4.2 系统仿真结果

实验一：视频流由YOLO模型处理的结果，如表 1所示。

表 1 YOLO视频流检测结果

数量	光照	摆放	平均置信度	识别率	置信度阈值	平均帧率
1	正常	规则	89%	100%	60%	13
1	正常	不规则	86%	100%	60%	12
1	弱	规则	84%	100%	70%	14
1	弱	不规则	82%	100%	70%	11
5	正常	规则	84%	100%	60%	11
5	正常	不规则	80%	80%	60%	12
5	弱	规则	78%	100%	70%	14
5	弱	不规则	75%	80%	70%	13
平均			82.25%	95%	65%	12.5

由表 1可见,基于YOLO的目标检测模块稳定运行，能够对场景中的目标物体进行有效识别与定位；实验结果表明，在不同数量、光照条件及摆放方式下，目标检测模块整体稳定。在单目标（数量为1）情况下，各实验条件下识别率均达到100%，平均置信度在82%~89%之间，平均帧率为10~15fps。

在多目标（数量为5）条件中，规则摆放条件下识别率仍可达到100%，不规则摆放条件下识别率下降至 80%。同时平均置信度也有所降低至范围75%~84%，置信度阈值在60%~70%。

在光照条件中，弱光环境下的平均置信度整体低于正常光照，置信度阈值在60%~70%。系统在不同条件下的平均帧率稳定在12~15fps之间。

实验二：物体由GraspNet模型平均20次的生成姿态结果，如表 2所示。

表 2 GraspNet生成的姿态结果

数量	光照	摆放	全部数量	候选数量	有效数量
1	正常	规则	132	74	2
1	正常	不规则	126	62	3
1	弱	规则	142	68	2
1	弱	不规则	138	58	3
5	正常	规则	154	70	12
5	正常	不规则	107	66	9
5	弱	规则	136	54	10
5	弱	不规则	154	61	8
平均			136.125	64.125	6.125

由表 2可见，基于GraspNet的抓取生成模块能够稳定运行，能够对场景中的目标物体生成合理的抓取候选并筛选出有效夹爪。实验结果表明，在不同物体数量、光照条件及摆放方式下的姿态生成模块整体运行稳定：

在单目标（数量为1）条件中，各实验条件下生成的抓取候选数量达50个以上，最终有效夹爪数量在2~3个。由数据可知有效夹爪的比例较低，但算法能够稳定输出可行抓取候选。

在多目标（数量为5）条件中，规则摆放条件有效夹爪数量最高可达12个，而不规则摆放条件下有效夹爪数量略有下降。由数据可知多目标场景下算法仍能生成较多合理夹爪，抓取有效性会降低。

在光照条件中，弱光环境下有效夹爪数量略低于正常光照，但通过候选筛选及评分机制，最终仍能生成合理抓取。

实验三：机械臂在GraspNet影响下的结果，如表 3所示。

表 3 机械臂是否使用候选姿态结果

数量	光照	摆放	不启用成功率	启用成功率
1	正常	规则	81%	83%
1	正常	不规则	83%	86%
1	弱	规则	82%	84%
1	弱	不规则	84%	83%
5	正常	规则	68%	56%
5	正常	不规则	56%	43%
5	弱	规则	64%	53%
5	弱	不规则	57%	42%
平均			71.875%	66.25%

由表 3可见，在目标数量、光照条件和摆放方式发生变化时，抓取姿态的影响也会产生变化。

在单目标（数量为1）条件中，抓取系统表现稳定，抓取姿态模块是否启用，成功率都维持在80%以上。相对于未启动抓取姿态，启用之后成功率会有小幅度提升。

在多目标物体（数量为5）条件中，系统抓取成功率整体下降。相对于未启动抓取姿态，启用之后成功率会有大幅下降。

在光照条件中，弱光环境的抓取成功率整体略低于正常光照，但变化幅度相对有限。对于是否启用抓取姿态的抓取成功率都基本没有影响。

在物体的摆放规则条件中，单个物体是否常规摆放对是否启用抓取姿态的成功率基本没有影响，但多个目标物体时，物体不规则摆放会大幅度降低抓取的成功率。规则摆放的抓取成功率整体高于不规则摆放，且在多目标场景中差异更加明显。

实验结果表明，在未启用抓取姿态模块时平均抓取成功率为 71.875%，启用抓取姿态后为 66.25%。在不同实验条件下均能够完成抓取任务，但在多目标及复杂场景下成功率有所下降。

实验四：机械臂在CCDIK驱动下的数据，如表 4所示。

表 4 机械臂运动结果

数量	距离	是否到达	轨迹平滑	抖动
1	近	是	基本平滑	无
1	远	是	基本平滑	无
5	5远	是	基本平滑	是
5	4远1近	是	基本平滑	是
5	3远2近	是	基本平滑	是
5	2远3近	是	基本平滑	是
5	1远4近	4个到达	基本平滑	是
5	5近	4个到达	基本平滑	是

由表 4可见，在不同目标数量与距离组合条件下，机械臂运动控制模块整体能够完成基本的运动任务。

在单个目标物体（数量为1）条件中，无论目标距离的远近，机械臂均能够成功到达目标物体的位置。各条件下机械臂运动轨迹整体均基本平滑，且不会出现抖动，

在多个目标物体（数量为5）条件中，机械臂能够完成分散的目标物体运动任务，但随着目标物体位置分布的变化，当多个物体贴合靠近时出现了未完全到达的情况。各条件下机械臂运动轨迹整体均基本平滑，但会出现抖动，

实验结果表明，机械臂运动控制模块在单目标场景里的表现稳定，能完成预期动作；在多目标条件下也具备执行能力，但存在部分问题，一方面是针对贴合的目标物体在执行时出现未达，另一方面是在运动过程中机械臂会出现抖动。

4.3 系统仿真分析

综合实验结果：

一，基于YOLO的目标检测模块在虚拟环境里稳定工作，可以较准确地识别出目标物体并给出位置信息。但实验过程存在一定局限性：例如当目标物体在几何形状上高度相似而仅在颜色特征上存在差异时，模型仍可能输出较高置信度，但分类结果存在偏差，表现出一定程度的误判现象。这表明当前模型对形状特征的判别能力相对不足，对颜色等低层视觉特征存在一定依赖。该问题可能与训练数据分布不均衡、特征表达能力有限与模型在训练过程中对判别性形状特征学习不足以及在特定

环境数据中过量学习产生拟合等现象有关。

综上，可以适当增加训练数据的数量和多样性，让模型多区分一些不同形状、不同颜色组合的物体；也可以在训练时强化对形状特征的学习，比如引入更多几何信息；同时再配合一些常见的方法，比如数据增强、正则化等，尽量减少过拟合的问题^{[19][20]}。

二，基于GraspNet的抓取姿态估计模块在虚拟环境中正常工作，可以通过深度图计算出一系列抓取位姿。在实验场景仅包含单个目标物体时，该模型在抓取姿态估计方面表现良好，成功率较高。但当场景扩展至多目标情形时，部分抓取结果出现偏差。该问题可能和抓取姿态生成的方式有关，当前方法依赖深度图来判断抓取位置，但深度图本身很难准确区分多个物体之间的边界，但两个目标物体挨得比较近、甚至有接触的情况下，模型可能把相邻的几个物体当成一个整体来看待，导致生成的抓取位置错误，从而影响机械臂的执行效果^[21]。

综上，可以在抓取前增加更细致的物体分割或者实例识别，从而把每个物体单独区分开来，再去做抓取姿态估计，这样可以获取到更加合理的抓取姿态。同时，如果能结合更多信息，而不只是依赖深度图，也有助于提升判断的准确性。

三，基于CCD的逆运动学控制模块在实验的过程基本稳定，在基本的旋转平移运动中可以优秀完成，但涉及路径规划和多关节配合会产生抖动。在抓取一些不规则形状物体的过程中，机械臂运动的轨迹不够平滑，末端执行器在转动的时候，会出现比较突兀的角度变化。

综上，机械臂运动稳定性不足主要源于两个方面：一方面，现有路径规划方法在复杂环境下的建模精度仍显不足，对多关节间的协同约束及障碍物规避策略考虑不够充分；另一方面，在关节运动过程中缺乏必要的平滑约束，例如插值精度不足、速度与加速度约束不够严格，导致轨迹连续性较差。因此可以在轨迹规划中给关节运动加一些平滑约束，或者引入更细致的动力学模型，让机械臂在复杂任务下的动作更稳定、更自然^[22]。

5 结论与展望

5.1 结论

本文围绕虚拟仿真环境下的机器人抓取问题，设计并实现了一套完整的机器人抓取系统。

本系统是在Unity平台搭建一个三维交互式仿真环境，再把目标检测、抓取姿态估计和机械臂运动控制三部分集成在一起，实现了识别目标、怎么抓目标、抓取目标的整套流程。具体实现的过程如下，首先使用训练的YOLO模型识别场景里的物体，并给出目标物体的位置。再使用GraspNet模型生成一批可行的抓取姿态方案。最后将信息传输到Unity运动控制模块，控制机械臂按照选定的姿态去执行动作，从而完成抓取。

综合实验结果表明，整个系统运行稳定，目标检测模块识别物体准确率较高，抓取姿态模块生成抓取姿态较合理，机械臂的运动控制正常。可以实现在虚拟环境对目标物体的抓取，证明了系统的可行性。

总体来看，本系统研究不仅验证了在虚拟仿真环境中开展机器人抓取研究的可行性，也为后续真实机器人系统的开发与优化提供了参考依据。

5.2 展望

本文已经将虚拟环境机械臂抓取系统设计完成，但仍然存在不足。例如训练模型的数据数量不多，后续优化模型时可以增加收集的数据。以及在面对一些摆放不规则的场景时存在运动轨迹不能一直维持在理想状态的问题，若在后续研究中引入人工智能方法，使机械臂能够在实际交互过程中自适应地调整抓取策略，则有望提升系统的运动自然性与对环境变化的适应能力。

本系统仍具备进一步扩展的空间，比如多机械臂一起协同抓取，然后一次性处理多个目标。或者添加实时反馈控制，让系统在运行的时候能一边仿真一边调试。

最重要的问题是本系统实验基于虚拟环境中实现，与现实环境存在差异。后续可以虚实结合，采用迁移学习方法面向仿真到现实场景，将仿真环境中训练得到的模型与控制策略迁移至真实系统中进行优化，使得虚拟环境的实现能得到现实环境的验证。

参考文献

- [1]. E. Todorov, T. Erez and Y. Tassa, "MuJoCo: A physics engine for model-based control," 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, Vilamoura-Algarve, Portugal, 2012, pp. 5026-5033 .
- [2]. Koenig, N., & Howard, A. (2004, September). Design and use paradigms for gazebo, an open-source multi-robot simulator. In 2004 IEEE/RSJ international conference on intelligent robots and systems (IROS)(IEEE Cat. No. 04CH37566) (Vol. 3, pp. 2149-2154). Ieee.
- [3]. Zhiyun Yin, Yujie Li, Overview of robotic grasp detection from 2D to 3D, Cognitive Robotics, Volume 2, 2022, Pages 73-82, ISSN 2667-2413.
- [4]. Guan, J., Hao, Y., Wu, Q., Li, S., & Fang, Y. (2024). A survey of 6dof object pose estimation methods for different application scenarios. Sensors, 24(4), 1076.
- [5]. 余吉雅,张艳超,张文博.基于YOLOv4-Tiny和RRT-Connect算法的机械臂自主抓取仿真[J].建模与仿真, 2023, 12(3): 2773-2781.
- [6]. 林邦,周伟,范江波,李锐.基于深度强化学习的机械臂视觉抓取控制优化方法[J].人工智能与机器人研究, 2018, 7(4): 200-206.
- [7]. 赵宇晴. 基于向量平均的6DOF估计值波动抑制方法研究[J]. 软件工程与应用, 2025, 14(3): 550-558.
- [8]. Hao-Shu Fang, Chenxi Wang, Minghao Gou, Cewu Lu; Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 11444-11453
- [9]. Andreas Aristidou, Joan Lasenby, FABRIK: A fast, iterative solver for the Inverse Kinematics problem, Graphical Models, Volume 73, Issue 5, 2011, Pages 243-260, ISSN 1524-0703.
- [10]. Guptasarma, S., Strong, M., Zhen, H., & Kennedy, M. D. (2025). J-PARSE: Jacobian-based Projection Algorithm for Resolving Singularities Effectively in Inverse Kinematic Control of Serial Manipulators. ArXiv, abs/2505.00306.
- [11]. 杨忠瑞,杨昌明,刘渝.基于改进遗传算法的工业机器人逆运动学求解[J].机械工程与技术, 2020, 9(5): 403-410.
- [12]. Yoon, J., Son, B., & Lee, D. (2023). Comparative Study of Physics Engines for

- Robot Simulation with Mechanical Interaction. *Applied Sciences*, 13(2), 680.
- [13]. Meier, A., Carroccio, S., Dornberger, R., & Hanne, T. (2021). Discussing the reality gap by comparing physics engines in kilobot simulations. *Journal of Robotics and Control (JRC)*, 2(5), 441-447..
- [14]. Manakitsa, N., Maraslidis, G. S., Moysis, L., & Fragulis, G. F. (2024). A review of machine learning and deep learning for object detection, semantic segmentation, and human action recognition in machine and robotic vision. *Technologies*, 12(2), 15.
- [15]. Xiaoming C, Tianzeng C, Haomin M, Ziqi Z, Dehua W, Jianchao S and Jun W (2024) An improved algorithm based on YOLOv5 for detecting *Ambrosia trifida* in UAV images. *Front. Plant Sci.* 15:1360419. doi: 10.3389/fpls.2024.1360419
- [16]. Varol-Malkoçoğlu, Ayşe & Şamlı, Rüya. (2026). A Detailed Review of YOLO Algorithms:YOLOv1 to YOLOv11. *ELECTRICA*. 26. 1-17.
- [17]. Asif, U., Tang, J., & Harrer, S. (2018, July). Graspnet: An efficient convolutional neural network for real-time grasp detection for low-powered devices. In *IJCAI* (Vol. 7, pp. 4875-4882)..
- [18]. 陈泳斌,程良伦.基于多物理特征的抓取检测方法[J].动力系统与控制, 2024, 13(2): 45-53.1
- [19]. Cao, Y., Xu, X., & Zhang, Y. (2025). Vision-Based Reinforcement Learning for Robotic Grasping of Moving Objects on a Conveyor. *Machines*, 13(10), 973..
- [20]. Gao, M., Cai, Q., Zheng, B., Shi, J., Ni, Z., Wang, J., & Lin, H. (2021). A hybrid yolov4 and particle filter based robotic arm grabbing system in nonlinear and non-gaussian environment. *Electronics*, 10(10), 1140.
- [21]. Mousavian, A., Eppner, C., & Fox, D. (2019). 6-dof graspnet: Variational grasp generation for object manipulation. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 2901-2910).
- [22]. 朝兴, 刘忠跃, 田龙淼, 郭伦, 王怡欣. 基于ROS-Unity的移动机器人虚实交互场景构建方法的研究[J]. 人工智能与机器人研究, 2020, 9(4): 217-231.

致谢

写到这里，也意味着本科学习阶段结束，时间过得真快，平日里在校的每个瞬间仍然历历在目。回想起在平日的校园生活中，受到了太多人的帮助，需要感谢的人太多太多。如果没有他们的支持和鼓励，也就没有现在的自己，在此谨向他们致以最由衷的感谢！

首先感谢老师的帮助。一开始选题的时候其实没方向，做设计的中间也卡过几次，在帮助下才慢慢理顺思路，最后拼出现在这一整套东西。还有一起做毕设的同学也帮了不少忙，平时随便聊聊各自做到哪了、哪里卡住了，有时候本来一个人死磕半天没结果，聊着聊着反而突然想通了。学校的条件也十分稳定，能让人安安静静把该做的东西一步步做出来。整体看下来，这次过程说轻松也不轻松，中间确实有点折腾，但换个角度想，也正是这些来回折腾的过程，让自己多学了点东西。所以还是想在这里简单说一句感谢，感谢所有在过程中给过帮助的人。

附录 仿真说明

重现系统实验仿真，需要配置两个Python环境,一个Unity环境。

YOLO的环境：

1. 打开YOLO文件夹。
2. 下载安装YOLO文件夹中的requirement.txt中的库。（`pip install -r requirement.txt`）

GraspNet的环境：

1. 打开GraspNet的文件夹。
2. 编译并安装Pointnet2。（`cd pointnet2; python setup.py install`）
3. 编译并安装Knn。（`cd knn; python setup.py install`）
4. 安装GraspNetAPI将它的包文件放置在GraspNet的环境中。
（`git clone https://github.com/graspnet/graspnetAPI.git; cd graspnetAPI; pip install` ）
5. 下载安装GraspNet文件夹中的requirement.txt的库。
（`cd docs; pip install -r requirements.txt; bash build_doc.sh`）

Unity的环境：

重现已有实验只需要打开Unity文件夹中把对应的场景打开即可

新建场景的配置：

1. 导入.FINAL-ROBOTIC_ARM_01.fbx。
2. 导入CCDIK.c#并绑定到机械臂上,填写对应参数。
3. 导入Capture.c#并绑定到机械臂上，填写对应参数。
4. 导入Finger.c#并绑定到机械臂夹子上，填写对应参数。
5. 创建抓取的物体，设置类别Capture。
6. 创建两个摄像机录入获取RGB-D图像，camera_depth需要设置层级1。
7. Unity中YOLO的接口分为三个：send.c#,get.c#,select.c#绑定到main摄像机中，并填写对应参数。
8. Unity中GraspNet的接口分为两个：EXR.c#,exr_get.c#，分别绑定到camera_depth与camera_RGB摄像机中，Unity还需要shader.shader作为深度摄像机的配置，拖至同级文件夹即可。
9. 运行YOLO后端与GraspNet后端再运行Unity项目。